Enrollment No.: 250843116002

Practical 4

AIM: Perform Exploratory Data Analysis (EDA) using descriptive statistics and identify trends in datasets.

Step 1: Import Libraries

```
#Import pandas library
import pandas as pd
#Import matplotlib library
import matplotlib.pyplot as plt
```

Step 2: Create Dataset

```
#create dataset.
df={
    "Month": ["January", "Februay", "March", "April", "May", "June", "Jully", "August","September", "October", "NMovember",
    "Units_Sold": [120, 135, 150, 145, 160, 175, 180, 170, 190, 210, 230, 250],
    "Revenue": [120000, 135000, 150000, 145000, 160000, 175000, 180000, 170000, 190000, 210000, 230000, 250000],
    "Profit": [20000, 23000, 52000, 24000, 27000, 30000, 31000, 29000, 33000, 37000, 41000, 45000]
}

df = pd.DataFrame(df)
#Print Dataframe
df
```

	Month	Units_Sold	Revenue	Profit
0	January	120	120000	20000
1	Februay	135	135000	23000
2	March	150	150000	52000
3	April	145	145000	24000
4	May	160	160000	27000
5	June	175	175000	30000
6	Jully	180	180000	31000
7	August	170	170000	29000
8	September	190	190000	33000
9	October	210	210000	37000
10	NMovember	230	230000	41000
11	December	250	250000	45000

Next steps: Generate code with df | New interactive sheet

Step 3: Inspect The Dataset

```
df.head()             #Display First Five rows.
```

	Month	Units_Sold	Revenue	Profit
0	January	120	120000	20000
1	Februay	135	135000	23000
2	March	150	150000	52000
3	April	145	145000	24000
4	May	160	160000	27000

Next steps: Generate code with df | New interactive sheet

```
df.shape                  #Display the no of rows and columns.
```

```
(12, 4)
```

```
df.columns                #Display name of All the columns.
```

```
Index(['Month', 'Units_Sold', 'Revenue', 'Profit'], dtype='object')
```

```
#Display info about dataset.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Month       12 non-null     object
 1   Units_Sold  12 non-null     int64
 2   Revenue     12 non-null     int64
 3   Profit      12 non-null     int64
dtypes: int64(3), object(1)
memory usage: 516.0+ bytes
```

Step 4: Generate Discriptive Statistic

```
df.describe()              #Display Statistics of Numeric Columns.
```

	Units_Sold	Revenue	Profit
count	12.000000	12.000000	12.000000
mean	176.250000	176250.000000	32666.666667
std	38.737168	38737.167963	9528.267355
min	120.000000	120000.000000	20000.000000
25%	148.750000	148750.000000	26250.000000
50%	172.500000	172500.000000	30500.000000
75%	195.000000	195000.000000	38000.000000
max	250.000000	250000.000000	52000.000000

Step 5: Claculate the avg no. of units sold per month

```
#Display Avg number of Units sold per month.
average_units = df["Units_Sold"].mean()
#DIsplay Result.
print("Average no. of units sold per month:", average_units)
```

```
Average no. of units sold per month: 176.25
```

Step 6: Calculate Median

```
median_units = df["Units_Sold"].median()            #Calculate the median.
#DIsplay Result.
print("Median no. of units sold per month:", median_units)
```

```
Median no. of units sold per month: 172.5
```

Step 7: Calculate Max Min

```
#Find thye highest no. of units sold in mounth.
maximun_units = df["Units_Sold"].max()

#Find thye highest no. of units sold in mounth.
minimun_units = df["Units_Sold"].min()

#Display Result.
print("Maximum no. of units sold in a month:", maximun_units)
print("Minimum no. of units sold in a month:", minimun_units)
```

```
Maximum no. of units sold in a month: 250
Minimum no. of units sold in a month: 120
```

Step 8: Calculate Standard Deviation

```
std_unit = df["Units_Sold"].std()                #Display SD
#Display the Result.
print("Standard Deviation:", std_unit)
```

```
Standard Deviation: 38.73716796331634
```

Step 9: Find thye mounth with Highest Sales

```
#Find the index of row having max revenue.
max_revenue_index = df["Revenue"].idxmax()
#Display the result (the index itself).
print("Row index with highest sales:", max_revenue_index)

#Retrieve the month corresponding to that index
month_with_highest_sales = df.loc[max_revenue_index, "Month"]

# Find highest_revenue value using the original numeric index
highest_revenue = df.loc[max_revenue_index, "Revenue"]

#Display results.
print("Month with highest sales:", month_with_highest_sales)
print("Highest revenue:", highest_revenue)
```

```
Row index with highest sales: 11
Month with highest sales: December
Highest revenue: 250000
```

Step 10: Find the Month with lowest sale

```
#Find the index of row having min revenue.
min_revenue_index = df["Revenue"].idxmin()
#Display the result (the index itself).
print("Row index with lowest sales:", min_revenue_index)

#Retrieve the month corresponding to that index
month_with_lowest_sales = df.loc[min_revenue_index, "Month"]

# Find lowest_revenue value using the original numeric index
lowest_revenue = df.loc[min_revenue_index, "Revenue"]

#Display results.
print("Month with lowest sales:", month_with_lowest_sales)
print("Lowest revenue:", lowest_revenue)
```

```
Row index with lowest sales: 0
Month with lowest sales: January
Lowest revenue: 120000
```

Step 11: Calculate total Revenue

```
#Diaplay total Revenue
total_revenue = df["Revenue"].sum()
#Display Result
print("Total Revenue:", total_revenue)
```

```
Total Revenue: 2115000
```

Step 12: Calculate the total Profit Generated per year

```
#Display Total Profit per yr
total_profit = df["Profit"].sum()
#Display Result
print("Total Profit:", total_profit)
```

```
Total Profit: 392000
```

Step 13: Identify the Sales Trend

```
#create a Line Chat to visualize monthly revenue.
plt.figure(figsize=(10, 5))

# Plot month on the x-axis and Revenue on the y-axis.
plt.plot(df["Month"], df["Revenue"], marker="o")

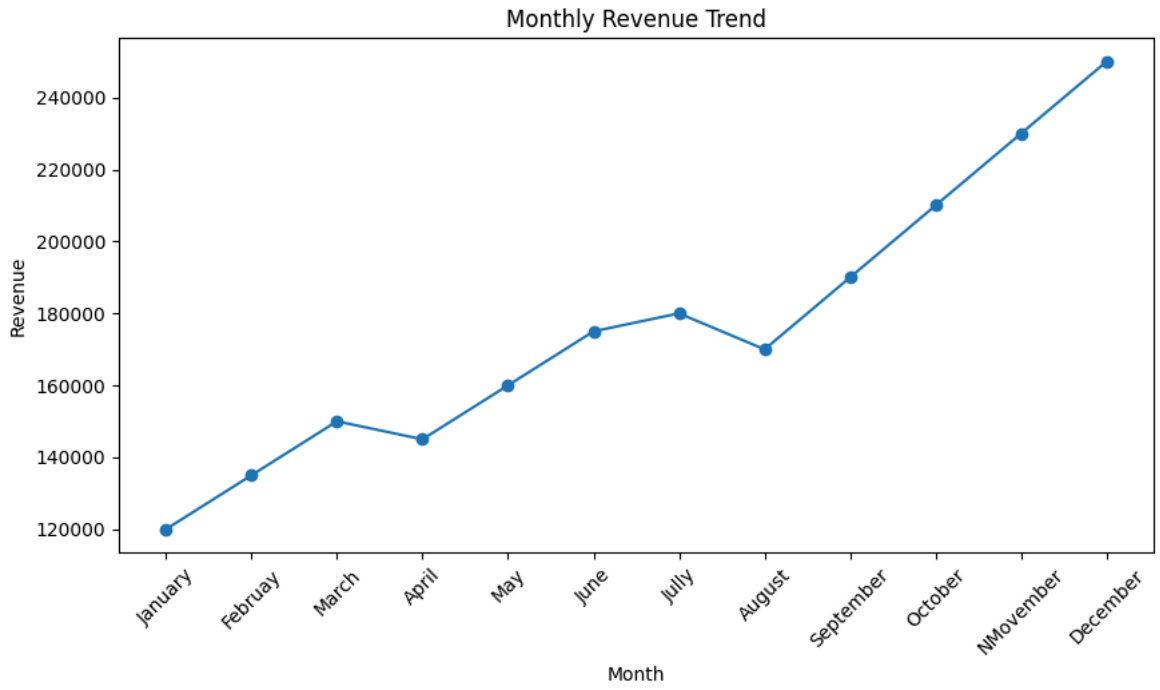
#Add a title on Graph.
plt.title("Monthly Revenue Trend")

#lable the X-axis.
plt.xlabel("Month")
```

```
#lable the Y-axis.
plt.ylabel("Revenue")

#Rotate Month name for batter readability.
plt.xticks(rotation=45)

#Display Graph.
plt.show()
```



Step 14: Analyze the Profit Trend

```
#Create a Line chart for Monthly Profit
plt.figure(figsize=(10, 5))

# Plot month on the x-axis and Profit on the y-axis.
plt.plot(df["Month"], df["Profit"], marker="o")

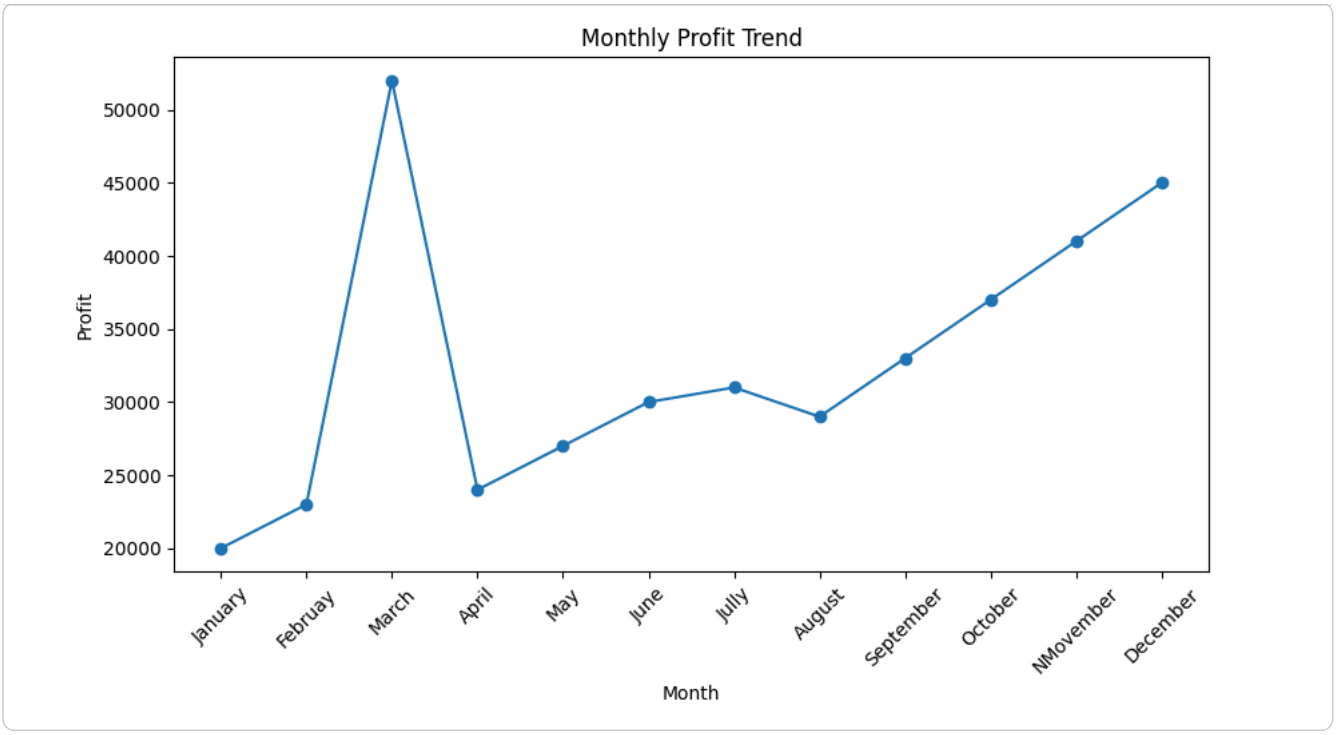
#Add a title on Graph.
plt.title("Monthly Profit Trend")

#lable the X-axis.
plt.xlabel("Month")

#lable the Y-axis.
plt.ylabel("Profit")

#Rotate Month name for batter readability.
plt.xticks(rotation=45)

#Display Graph.
plt.show()
```



Step 15: Compare Revenue And Profit

```
#Create a Line chart to compare Revenue and Profit.
plt.figure(figsize=(10, 5))

#Plot Revenue.
plt.plot(df["Month"], df["Revenue"], marker="o", label="Revenue")

#Plot Profit.
plt.plot(df["Month"], df["Profit"], marker="o", label="Profit")

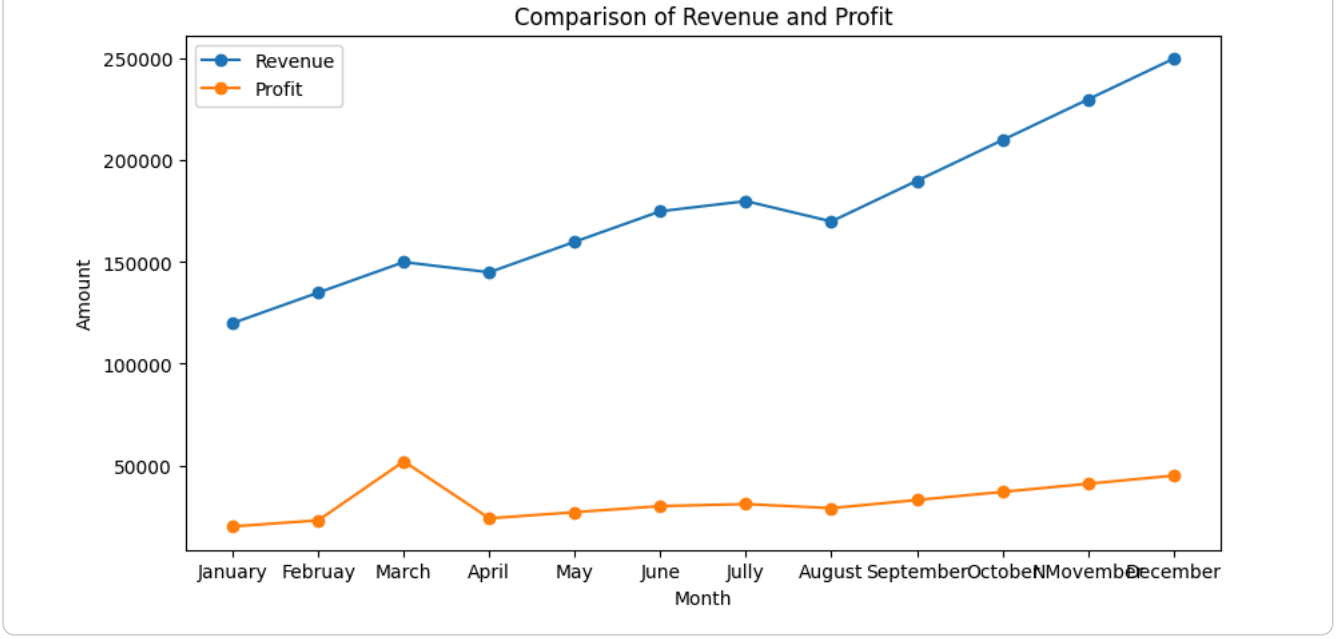
#Add a title on Graph.
plt.title("Comparison of Revenue and Profit")

#lable the X-axis.
plt.xlabel("Month")

#lable the Y-axis.
plt.ylabel("Amount")

#Display lagend to identify two lines.
plt.legend()

#Display Graph.
plt.show()
```



Step 16:Identify Growth in Sales

```
#Calculate the % change in  Revenue from 1 month to next.
df["Revenue_Growth_%"] = df["Revenue"].pct_change() * 100

#DIsplay the Updated Dataset.
df
```

	Month	Units_Sold	Revenue	Profit	Revenue_Growth_%
0	January	120	120000	20000	NaN
1	Februay	135	135000	23000	12.500000
2	March	150	150000	52000	11.111111
3	April	145	145000	24000	-3.333333
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	Jully	180	180000	31000	2.857143
7	August	170	170000	29000	-5.555556
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	NMovember	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Next steps: Generate code with df | New interactive sheet

Step 17: Identify the Month With Increasing Revenue

```
#Filter Dataset to display only these months where revenue
increasing_sales = df[df["Revenue_Growth_%"]>0]
#DIsplay the Updated Dataset.
increasing_sales
```

	Month	Units_Sold	Revenue	Profit	Revenue_Growth_%
1	Februay	135	135000	23000	12.500000
2	March	150	150000	52000	11.111111
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	Jully	180	180000	31000	2.857143
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	NMovember	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Next steps: Generate code with increasing_sales | New interactive sheet

Step 18: Save the EDA Result

```
df.to_csv("EDA_Sales_Analysis.csv", index=False)          #Save EDA Result

print("CSV file saved Successfully")
```

```
CSV file saved Successfully
```

Step 19: Download CSV In Google colabe

```
from google.colab import files                    #Download CSV file

files.download("EDA_Sales_Analysis.csv")
```